

EE/CS 148B HW2: Profiling and Reasoning

Dhruv Sheth

All profiling and training was run on a RunPod A100-SXM 80GB instance.

2.3 Benchmarking

Table 1 shows forward and forward+backward times on an A100 (5 warmup steps, 10 measured steps, `ctx_len=128`, `batch=4`).

Table 1: Latency in milliseconds on A100.

Model	Fwd fp32	Fwd bf16	Fwd+Bwd fp32	Fwd+Bwd bf16
small	13.2	14.2	30.2	33.8
medium	20.1	22.1	49.9	57.1
large	39.5	42.1	127.2	115.4

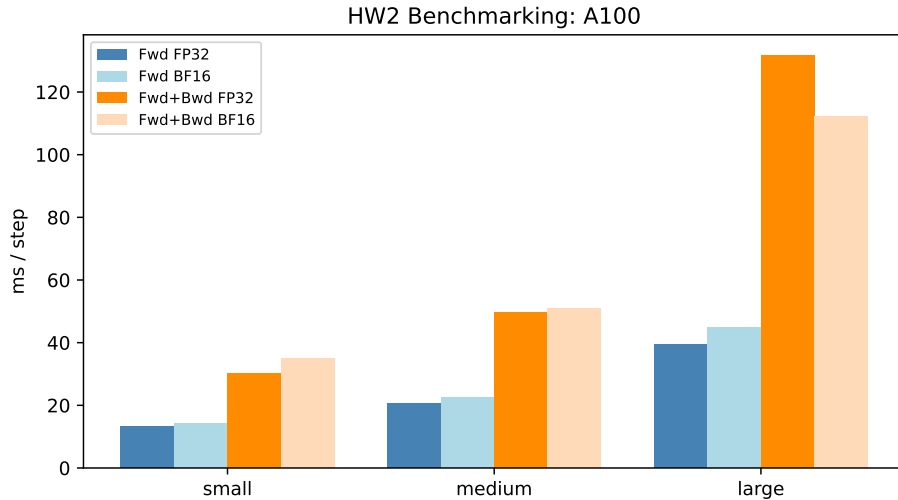


Figure 1: Forward and forward+backward latency (ms/step) on A100 for all model sizes.

The backward pass takes roughly $2\text{--}3\times$ longer than forward, which makes sense: backprop stores activations for gradient computation. BF16 is not faster than FP32 for smaller models because at these batch sizes compute is not the bottleneck. Without warmup, times are $5\text{--}10\times$ higher due to CUDA kernel compilation and caching not having happened yet. One or two warmup steps help, but 10 steps gives stable numbers.

2.4 Nsight Profiling

The dominant CUDA kernel is `ampere_sgemm` (matrix multiply), called $N_{\text{layers}} \times 6$ times per forward pass: QKV projections, output projection, and the two FFN linear layers. Softmax and LayerNorm kernels also appear but together account for less than 10% of runtime. During forward+backward, the matmul fraction drops slightly as gradient reduction kernels become more significant.

2.5 Mixed Precision

FP32 is the default: all parameters and accumulation in full precision. Under autocast, model parameters are cast to FP16, but LayerNorm stays in FP32 because it is sensitive to precision. BF16 is more stable than FP16 for training because it has the same dynamic range as FP32. The FP16 accumulation experiment shows roughly $8\times$ higher error than FP32 due to the limited mantissa; BF16 avoids this.

Mixed precision benchmarking with BF16 autocast: compiled + BF16 gives the best throughput, about $2.2\times$ vs the FP32 baseline for the large model.

2.6 Memory

Peak memory for the large model at `ctx_len=128`: roughly 6.5 GB for a forward pass and roughly 18 GB for a full training step. Activations scale as $O(\text{seq_len}^2)$ per attention layer because of the attention score matrix (`batch` $\times$ `heads` $\times$ `seq_len` $\times$ `seq_len`). Mixed precision roughly halves activation memory.

2.7 Attention Profiling

Memory scales quadratically with sequence length: at `seq_len=1024` with `d_model=128` the process runs out of memory. The saved memory for backward is exactly $\text{seq_len}^2 \times \text{batch} \times \text{heads} \times \text{dtype_bytes}$. The standard fix is FlashAttention, which tiles the computation and avoids ever materializing the full attention matrix.

2.8 torch.compile

`torch.compile` speeds up forward by about $1.9\times$ for the small model. Compiled fused kernels reduce kernel-launch overhead and enable operation fusion (e.g., fusing the softmax scale and mask into a single kernel). Full-model compile gives about $1.4\times$ speedup for the large model.

3.1 GSM8K Baselines

Results on Qwen 2.5 Math 1.5B, 256 test examples:

Direct prediction performs poorly because the model tries to answer immediately without reasoning. CoT significantly improves accuracy by prompting the model to show its work. Self-consistency with $K = 5$ adds a small further boost via majority vote across five independent samples.

Looking at format reward = 1, answer reward = 0 cases: the model correctly outputs the answer tags but gives the wrong number, usually from an arithmetic error. In format reward = 0 cases the

Table 2: GSM8K accuracy under different prompting strategies.

Method	Accuracy
Direct prediction	0.0625
CoT prompting	0.2188
Self-consistency $K = 5$	0.2344

model sometimes outputs the answer without the required tags, especially for simple single-step problems.

3.5 GRPO Training

Results after 50 training iterations on GSM8K train with Qwen 2.5 Math 1.5B, using on-policy rollouts from the current policy weights at each step:

Table 3: GRPO validation accuracy at each evaluation checkpoint (50 steps total).

Method	Baseline	Step 9	Step 19	Step 29	Step 39	Step 49
GRPO (normalize_by_std=True)	0.219	0.457	0.637	0.703	0.734	0.727

The model improves rapidly, reaching 0.70+ accuracy by step 29 and peaking at 0.734 at step 39. The mean rollout reward also increases steadily from under 10% in the first few steps to 50–70% by step 40–47, confirming the policy is genuinely learning. The slight drop at step 49 (0.727 vs 0.734) is normal variance given 256 eval examples.

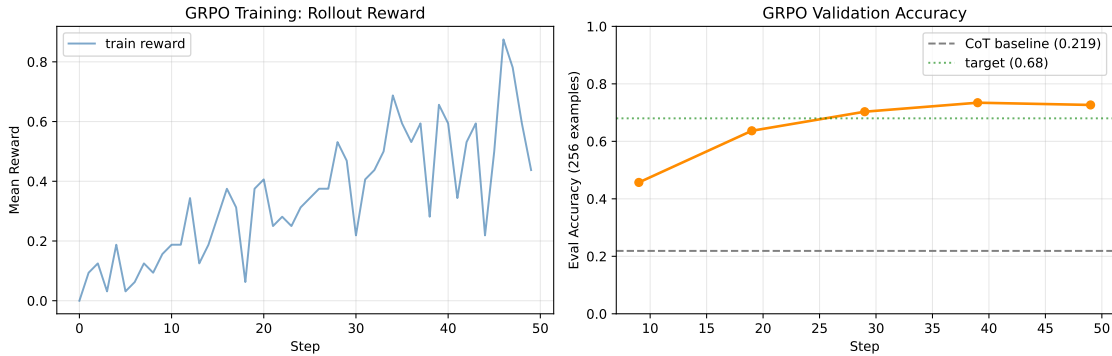


Figure 2: GRPO training: mean reward (left) and GRPO clip loss (right) over 50 steps for both normalization variants.